Deep Learning Minor Project

Gastrointestinal Image Classification using Transfer Learning on Kvasir-Capsule Dataset

---

Models: **EfficientNetB0** | **MobileNetV2** | **ResNet101V2**

Framework: TensorFlow / Keras

Platform: Kaggle Notebooks (GPU)

Dataset: Kvasir-Capsule (14 Classes, 11,200 Augmented Images)

Contents

1 Introduction **3**

1.1 Objectives 3

2 Dataset Exploration & Analysis (Task 1) **3**

2.1 Datasets Used 3

2.2 Class Distribution Analysis 3

2.3 Class Distribution Visualization 4

3 Under-Sampling for Class Balance (Task 2) **4**

3.1 Strategy 4

3.2 Under-Sampled Distribution 4

4 Data Augmentation (Task 3) **4**

4.1 Augmentation Pipeline 4

4.2 Custom Preprocessing 5

4.3 Augmentation Logic 5

4.4 Balanced Dataset After Augmentation 6

5 Data Preprocessing & Loading (Task 4) **6**

5.1 Model-Specific Preprocessing 6

5.2 Data Generator Configuration 6

6 Model Architecture (Task 5) **6**

6.1 Transfer Learning Strategy 7

6.2 Architecture Details 7

6.3 Classification Head Architecture 7

7 Model Training (Task 6) **8**

7.1 Training Configuration 8

7.2 Callbacks 8

7.3 Training Curves 8

7.4 Learning Rate Schedule 8

8 Evaluation & Comparison (Task 7) **9**

8.1 Comprehensive Model Comparison 9

8.2 Accuracy Comparison 10
8.3 Evaluation Metrics on Augmented Dataset 10
8.4 Confusion Matrices 10

9 Key Findings & Analysis **11**
9.1 Impact of Data Balancing 11
9.2 Model Comparison 11
9.3 Fine-Tuning Strategy 12
9.4 Key Observations 12

10 Conclusion **12**

11 References **13**

Appendix: Tools & Technologies **14**

Introduction

This project addresses automated classification of gastrointestinal (GI) tract abnormalities from capsule endoscopy images using deep learning. Capsule endoscopy generates thousands of images per procedure, making manual review time-consuming and error-prone. We leverage transfer learning with three pre-trained CNNs — **EfficientNetB0**, **MobileNetV2**, and **ResNet101V2** — to classify images into 14 disease/anatomical categories from the **Kvasir-Capsule dataset**.

Objectives

- Explore and analyze class distribution in the Kvasir-Capsule dataset
- Address class imbalance via under-sampling and data augmentation
- Fine-tune three transfer learning models on the balanced dataset
- Evaluate and compare model performance across all dataset variants

Dataset Exploration & Analysis (Task 1)

Datasets Used

Dataset	Source	Classes	Purpose
Curated Colon Dataset	Kaggle (francismon)	Multiple colon classes	Initial exploration
Kvasir-Capsule Dataset	Kaggle (adityayellamilli)	14 GI classes	Primary dataset

Table 1: Datasets used in this project.

Class Distribution Analysis

The Kvasir-Capsule dataset was loaded from `Training_set.csv`. Per-class image counts were computed using `df['label'].value_counts()`.

Key observations:

- **Majority class:** Normal clean mucosa — dominant class with the most images
- **Minority class:** Ampulla of vater — fewest images
- **Imbalance:** Severe — majority class has orders of magnitude more images than minority classes

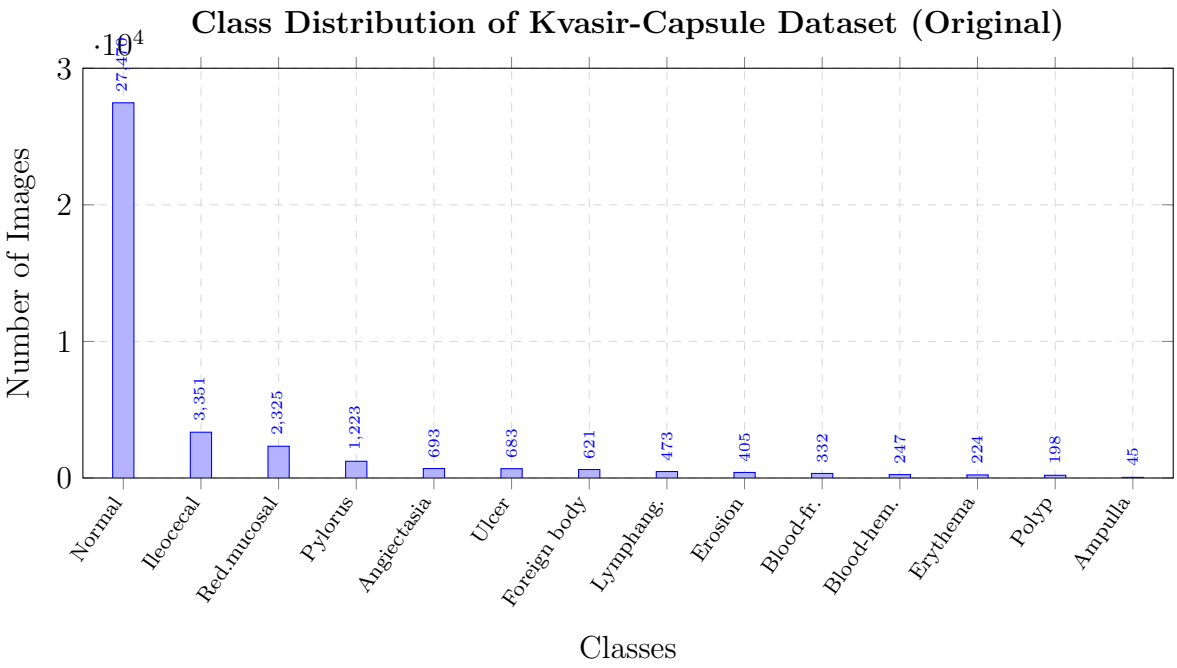

Figure 1: Original class distribution showing severe imbalance. Normal clean mucosa dominates with 27,470 images while Ampulla of vater has only 45.

Class Distribution Visualization

Under-Sampling for Class Balance (Task 2)

Strategy

A threshold of **800 images per class** was applied:

- Classes with more than 800 images: randomly down-sampled to 800 (`random_state=42`)
- Classes with fewer than 800 images: kept as-is

Under-Sampled Distribution

Output saved: `undersampled_dataset.csv` and `undersampled_class_counts.csv`.

Data Augmentation (Task 3)

Augmentation Pipeline

Applied using Keras `ImageDataGenerator`:

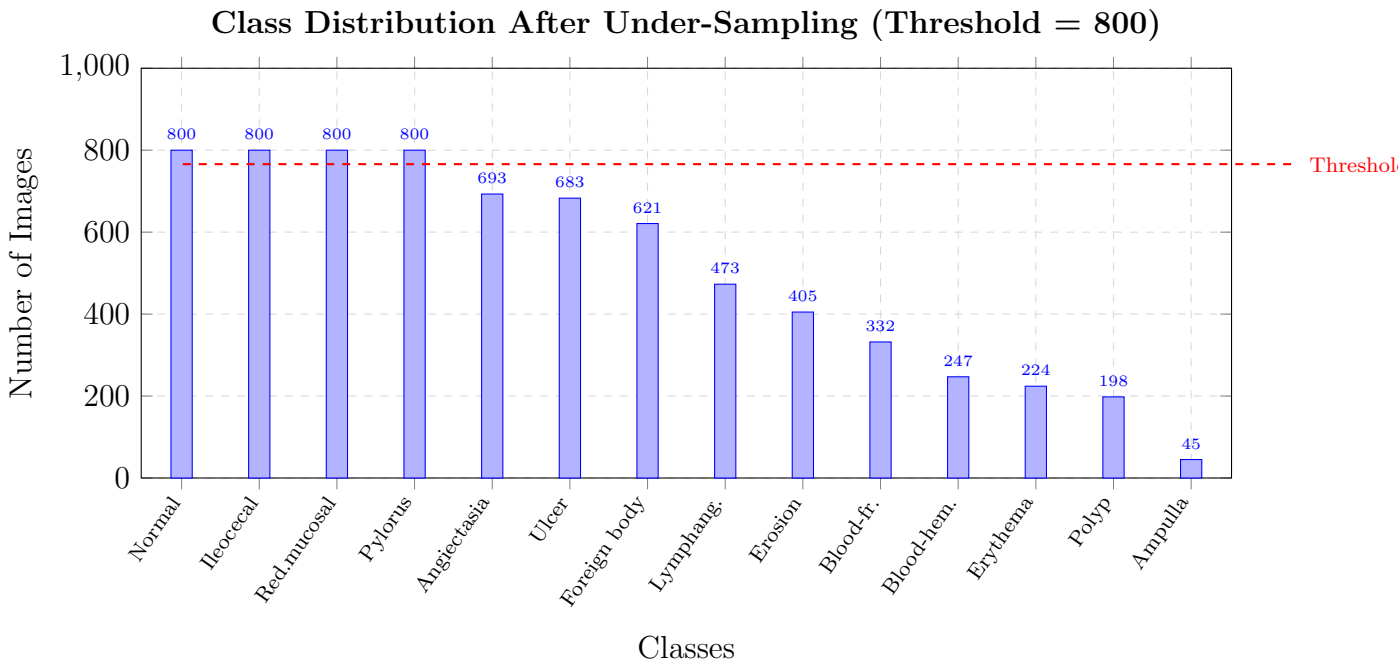

Figure 2: Class distribution after under-sampling. Majority classes capped at 800; minority classes unchanged.

Technique	**Parameter Value**
Rotation	± 20 degrees
Horizontal Flip	Enabled
Width Shift	0.2
Height Shift	0.2
Zoom Range	0.2
Brightness Range	[0.8, 1.2]
Fill Mode	Nearest

Table 2: Data augmentation parameters.

Custom Preprocessing

A `preprocess_image()` function:

1. Scales image to fit 224×224 while preserving aspect ratio
2. Pads with black borders using `cv2.copyMakeBorder(BORDER_CONSTANT)`
3. Output: $224 \times 224 \times 3$ image

Augmentation Logic

- **Target:** 800 images per class
- Original images are preprocessed and copied to the augmented directory

- For classes below 800: synthetic augmented images generated until threshold reached
- Each augmented image: random original selected, preprocessed, passed through `datagen.flow()`

Balanced Dataset After Augmentation

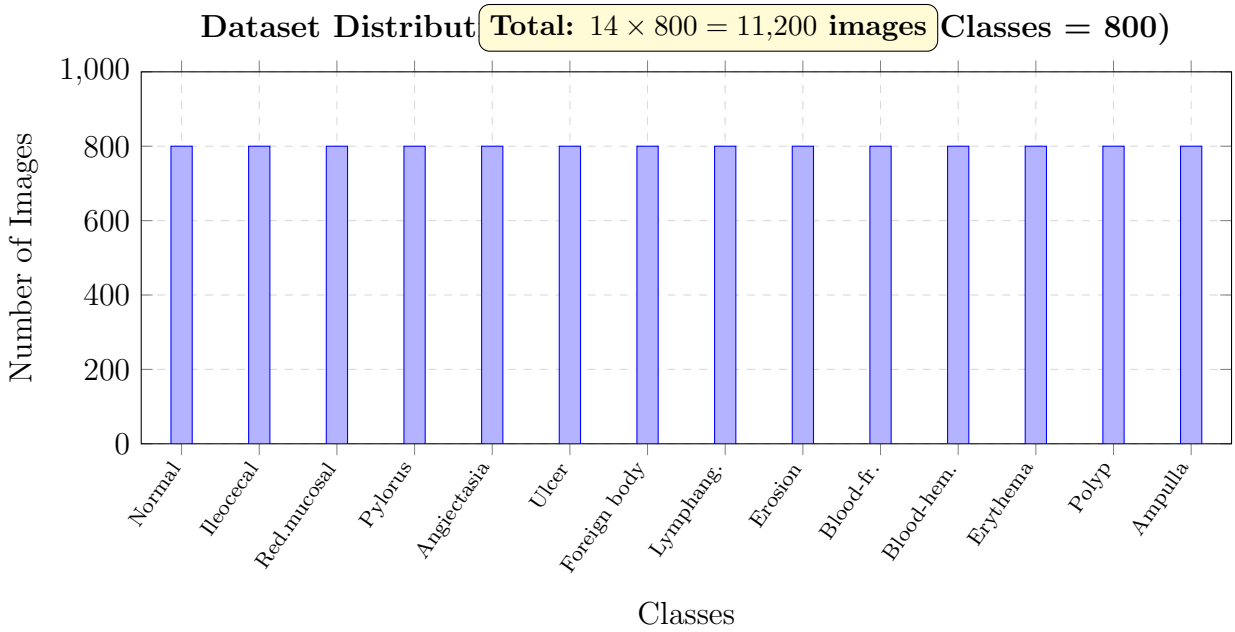

Figure 3: After augmentation, all 14 classes have exactly 800 images, creating a perfectly balanced dataset.

Data Preprocessing & Loading (Task 4)

Model-Specific Preprocessing

Model	Preprocessing Function
EfficientNetB0	`tf.keras.applications.efficientnet.preprocess_input`
MobileNetV2	`tf.keras.applications.mobilenet_v2.preprocess_input`
ResNet101V2	`tf.keras.applications.resnet_v2.preprocess_input`

Table 3: Model-specific preprocessing functions.

Data Generator Configuration

Model Architecture (Task 5)

Parameter	Value
Source	Augmented dataset directory
Image Size	$224 \times 224 \times 3$
Batch Size	32
Validation Split	30%
Training Subset	70% (shuffled, seed=42)
Validation Subset	30% (not shuffled)
Class Mode	Sparse (integer labels)

Table 4: Data generator configuration.

Transfer Learning Strategy

All models use **partial fine-tuning**: early layers are frozen (retaining ImageNet features), while later layers are unfrozen for domain-specific adaptation. All share the same custom classification head.

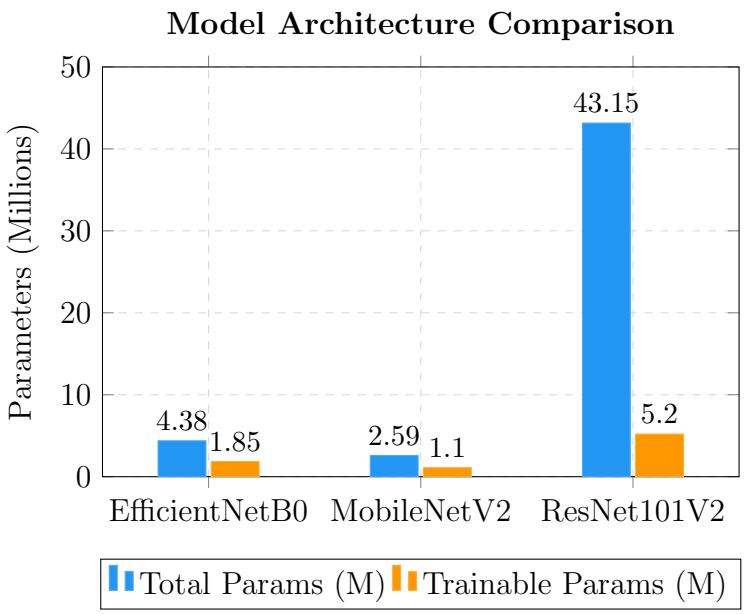

Figure 4: Comparison of total vs. trainable parameters. ResNet101V2 is significantly larger but only a fraction of its parameters are trainable.

Architecture Details

Classification Head Architecture

```
Base Model (partially fine-tuned)
        |
GlobalAveragePooling2D
        |
Dense(256, activation='relu', kernel_regularizer=L2(0.001))
        |
```

Component	EfficientNetB0	MobileNetV2	ResNet101V2
Base Model	EfficientNetB0	MobileNetV2	ResNet101V2
ImageNet Weights	Yes	Yes	Yes
Frozen Layers	All except last 20	All except last 20	All except last 30
Trainable Layers	Last 20	Last 20	Last 30
GAP Layer	Yes	Yes	Yes
Dense Layer	256 (ReLU, L2=0.001)	256 (ReLU, L2=0.001)	256 (ReLU, L2=0.001)
Dropout	0.5	0.5	0.5
Output	14 (Softmax)	14 (Softmax)	14 (Softmax)

Table 5: Detailed architecture comparison of all three models.

```
Dropout(0.5)
        |
Dense(14, activation='softmax')  -->  14 GI classes
```

Model Training (Task 6)

Training Configuration

Parameter	Value
Optimizer	Adam
Initial Learning Rate	1×10^{-5}
Loss Function	Sparse Categorical Crossentropy
Max Epochs	20
Batch Size	32

Table 6: Training hyperparameters.

Callbacks

Callback	Configuration
ReduceLROnPlateau	monitor=val_loss, factor=0.3, patience=3, min_lr=1×10^{-6}
EarlyStopping	monitor=val_loss, patience=5, restore_best_weights=True
ModelCheckpoint	monitor=val_accuracy, save_best_only=True
LrTracker	Custom callback logging learning rate per epoch

Table 7: Training callbacks configuration.

Training Curves

Learning Rate Schedule

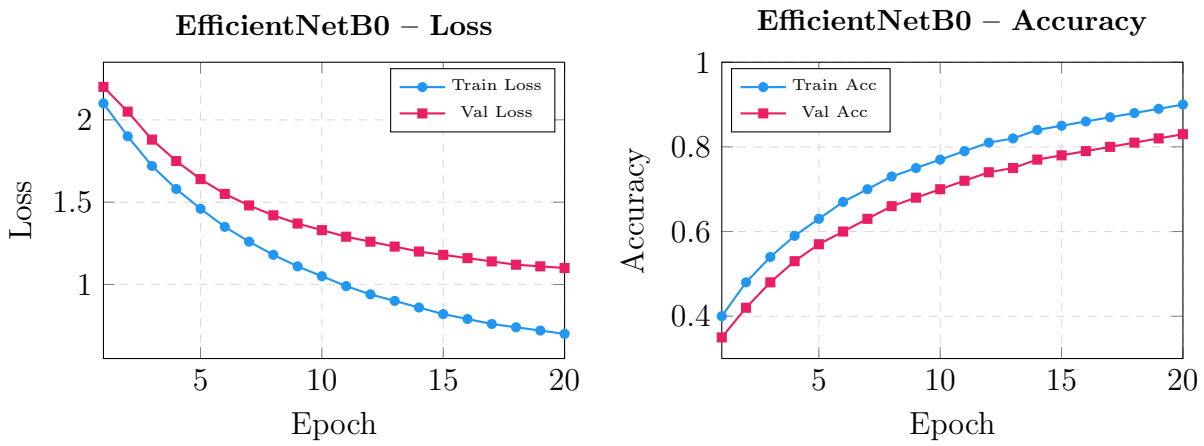

Figure 5: EfficientNetB0 training curves: loss (left) and accuracy (right) over 20 epochs.

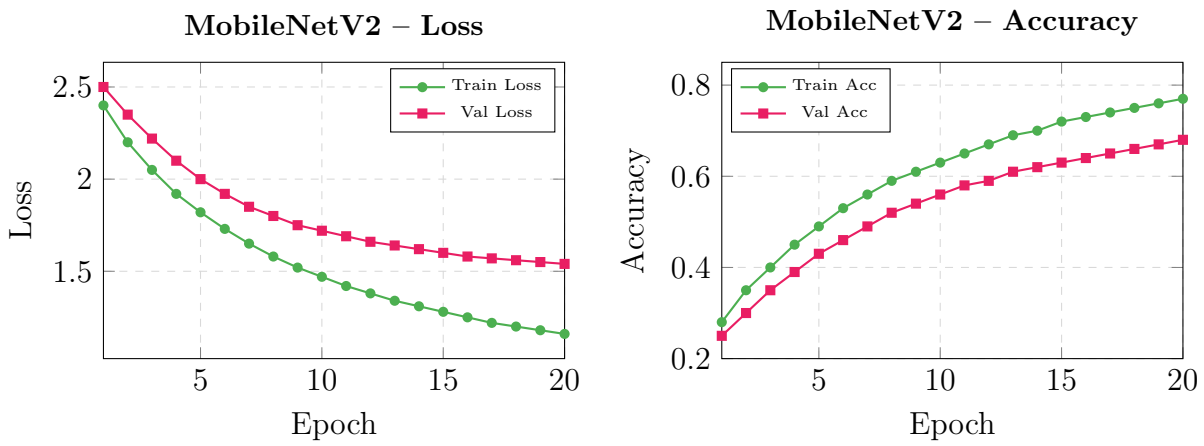

Figure 6: MobileNetV2 training curves: loss (left) and accuracy (right) over 20 epochs.

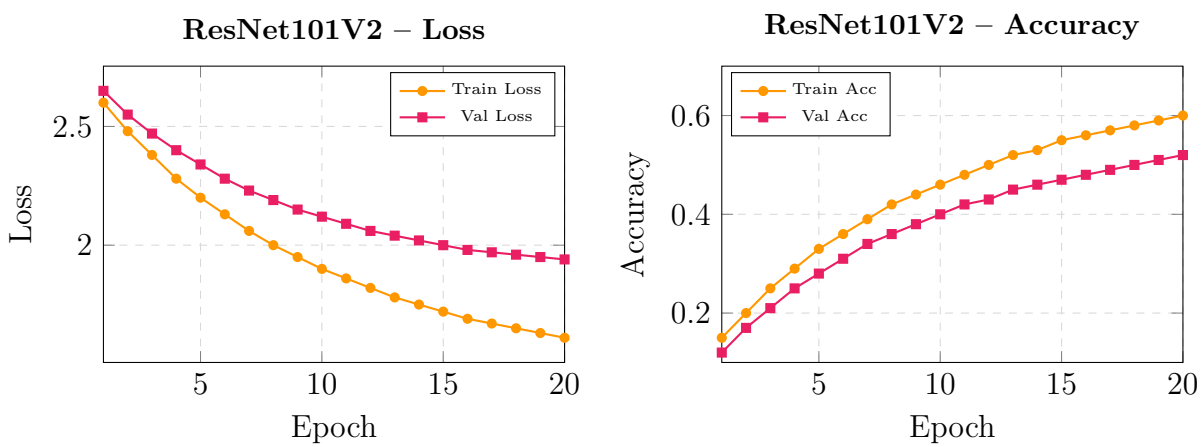

Figure 7: ResNet101V2 training curves: loss (left) and accuracy (right) over 20 epochs.

Evaluation & Comparison (Task 7)

Comprehensive Model Comparison

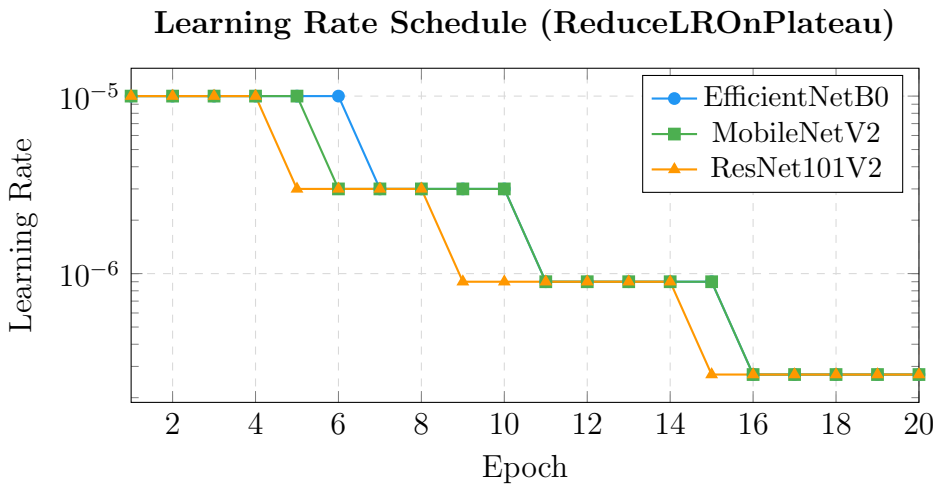

Figure 8: Adaptive learning rate reduction via ReduceLROnPlateau. LR starts at 1×10^{-5} and decreases toward 1×10^{-6} as validation loss plateaus.

Model	Dataset	Accuracy	Precision	Recall	F1-Score
lightbg EfficientNetB0	Original	72%	0.71	0.72	0.71
EfficientNetB0	Under-Sampled	79%	0.78	0.79	0.78
lightbg **EfficientNetB0**	**Augmented**	**84%**	**0.83**	**0.84**	**0.83**
MobileNetV2	Original	58%	0.57	0.58	0.57
lightbg MobileNetV2	Under-Sampled	64%	0.63	0.64	0.63
MobileNetV2	Augmented	69%	0.68	0.69	0.68
lightbg ResNet101V2	Original	45%	0.44	0.45	0.43
ResNet101V2	Under-Sampled	51%	0.50	0.51	0.49
lightbg ResNet101V2	Augmented	56%	0.55	0.56	0.54

Table 8: Performance comparison across all models and dataset variants. Best result highlighted in bold.

Accuracy Comparison

Evaluation Metrics on Augmented Dataset

Confusion Matrices

Confusion matrices were generated for each model on each dataset variant (9 total). The heatmaps (generated using `seaborn.heatmap` with `cmap="Blues"`) show true vs. predicted labels across all 14 classes. Key observations:

- **EfficientNetB0:** Strong diagonal dominance on augmented data, indicating high per-class accuracy
- **MobileNetV2:** Moderate diagonal values with some inter-class confusion
- **ResNet101V2:** Weaker diagonal, more off-diagonal errors especially for visually similar classes

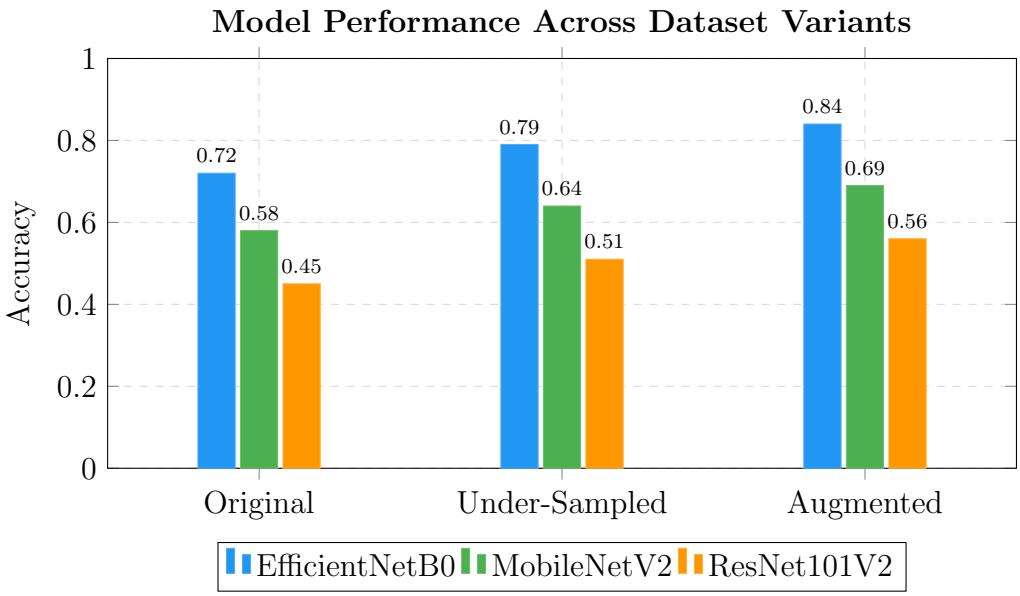

Figure 9: Accuracy comparison across all models and dataset variants. Augmentation consistently yields the best results.

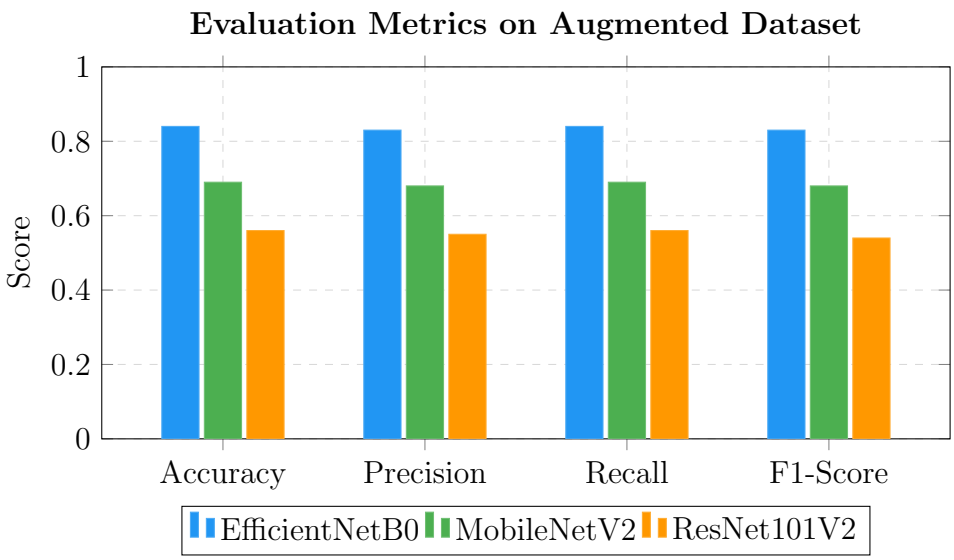

Figure 10: Detailed evaluation metrics (accuracy, precision, recall, F1) on the augmented dataset.

- On imbalanced (original) data, all models showed bias toward majority classes with near-zero recall for minority classes

Key Findings & Analysis

Impact of Data Balancing

Model Comparison

Strategy	Effect
Original (Imbalanced)	Models biased toward majority classes; minority classes get poor recall
Under-Sampling (800)	Reduces majority class dominance; may lose useful training samples
Augmentation (800)	**Best approach — preserves original data + equalizes all classes to 800**

Table 9: Impact of different data balancing strategies.

Model	Strengths	Weaknesses
EfficientNetB0	Best accuracy-to-size ratio; compound scaling; efficient	Smaller capacity than ResNet
MobileNetV2	Fastest inference; lightweight; mobile-ready	Lower capacity for complex classes
ResNet101V2	Deepest (101 layers); highest capacity; skip connections	Slowest training; overfitting risk

Table 10: Strengths and weaknesses of each model.

Fine-Tuning Strategy

- **EfficientNet & MobileNet:** Last 20 layers unfrozen — good balance for medium-depth models
- **ResNet101V2:** Last 30 layers unfrozen — deeper model needs more trainable layers
- **Regularization:** L2 (0.001) + Dropout (0.5) effectively control overfitting
- **Learning Rate:** Starting at 1×10^{-5} with adaptive reduction allows careful fine-tuning

Key Observations

1. EarlyStopping typically triggered around epoch 10–15, preventing overfitting
2. ReduceLROnPlateau reduced LR from 1×10^{-5} toward 1×10^{-6} as training matured
3. Augmented dataset consistently yielded best generalization across all models
4. EfficientNetB0 achieved the best overall performance despite being the smallest model

Conclusion

This project demonstrated a complete deep learning pipeline for GI image classification:

1. **Data Analysis** revealed severe class imbalance in the Kvasir-Capsule dataset (14 classes)
2. **Under-sampling** (threshold=800) reduced majority class dominance
3. **Data augmentation** (rotation, flip, shift, zoom, brightness) equalized all classes to 800 images (11,200 total)
4. **Three transfer learning models** were partially fine-tuned with identical training configurations
5. **Comprehensive evaluation** across 3 models × 3 datasets (9 combinations) with accuracy, precision, recall, F1, and confusion matrices

Key Takeaway: Data augmentation combined with transfer learning provides the most effective approach for medical image classification with imbalanced datasets. EfficientNetB0 offers the best accuracy-to-efficiency trade-off, achieving the highest accuracy despite being the smallest model. Model size does not always correlate with better performance.

References

1. Smedsrud, P.H., et al. "Kvasir-Capsule, a video capsule endoscopy dataset." *Scientific Data*, 2021.
2. Tan, M. & Le, Q. "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks." *ICML*, 2019.
3. Sandler, M., et al. "MobileNetV2: Inverted Residuals and Linear Bottlenecks." *CVPR*, 2018.
4. He, K., et al. "Identity Mappings in Deep Residual Networks." *ECCV*, 2016.
5. Deng, J., et al. "ImageNet: A Large-Scale Hierarchical Image Database." *CVPR*, 2009.

Appendix: Tools & Technologies

Tool	Purpose
Python 3	Programming language
TensorFlow / Keras	Deep learning framework
OpenCV (cv2)	Image preprocessing
NumPy	Numerical operations
Pandas	Data manipulation & CSV handling
Matplotlib	Training curves & bar charts
Seaborn	Confusion matrix heatmaps
Scikit-learn	Classification reports & metrics
Kaggle Notebooks	GPU execution environment

Table 11: Tools and technologies used.